Discussion of Exercise Sheet 7

Exercise 7.4 *String counting* **(2 bonus points).** → Peergrading Aufgabe für heute

Given a binary string $S \in \{0,1\}^n$ of length n, let $f(S)$ be the number of times "11" occurs in the string, i.e. the number of times a 1 is followed by another 1. In particular, the occurrences do not need to be disjoint. For example $f(\text{“}\underline{11}10\underline{11}\text{”}) = 3$ because the string contains three underlined 1s that are followed by another 1. Given n and k, the goal is to count the number of binary strings S of length n with $f(S) = k$.

Describe a DP algorithm that, given positive integers n and k with $k < n$, reports the required number. Your solution should have complexity at most $O(nk)$.

Dimension: DP[1...n][0...k][0..1]

Teilproblem: DP[i, j, ℓ] := number of strings of length i with j occurences of "11" that end in ℓ

Base Case:

DP[1, 0, 0] = 1 → the string "0"

DP[1, 0, 1] = 1 → the string "1"

DP[1, j, 0] = 0 für 1 ≤ j ≤ k → a string of length 1 cannot contain "11"

DP[1, j, 1] = 0 für 1 ≤ j ≤ k → a string of length 1 cannot contain

Rekursion:

Fall ℓ = 0: DP[i, j, 0] = DP[i - 1, j, 0] + DP[i - 1, j, 1]

alle strings der Länge i-1 mit j mal "11" und beliebiger Endung

Fall ℓ = 1: DP[i, j, 1] = DP[i - 1, j, 0] + DP[i - 1, j - 1, 1]

:= 0 falls j = 0

die 1 gehört nicht zu einem "11" Paar

die 1 gehört zu einem "11" Paar

Berechnungsreihenfolge:

```
for i = 1 .. n
    for j = 0 .. k
        for ℓ = 0 .. 1: berechne DP[i, j, ℓ]
```

Lösung: DP[n, k, 0] + DP[n, k, 1]

Laufzeit: Θ(nk)

Graphen

Graph $G = (V, E)$ mit $n = |V|$ Knoten und $m = |E|$ Kanten

Knotenmenge / vertices / nodes V, darf nicht leer sein. oft ist $V = \{1, 2, \dots, n\}$

Kantenmenge / edges / links: E wobei $E \subseteq \{ \{u, v\} \mid u \in V, v \in V, u \neq v \}$

- $\{u, v\}$: Kante zwischen u und v
- $u \neq v$: es darf keine Schleifen geben

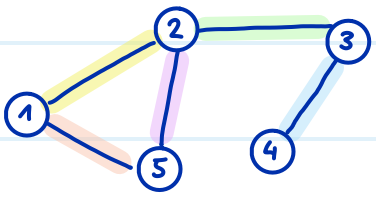
Benachbart / adjacent Knoten u und v sind benachbart, falls $\{u, v\} \in E$

Anliegend / incident: Kante $\{u, v\}$ ist zu u und v anliegend

Grad / degree: $\deg(v)$ = Anzahl Kanten, die zu v anliegen

Bsp: $V = \{1, 2, 3, 4, 5\}$

$E = \{\{1,2\}, \{2,3\}, \{3,4\}, \{2,5\}, \{1,5\}\}$



→ 2 und 3 sind benachbart

→ $\{3,4\}$ ist zu 3 und 4 anliegend

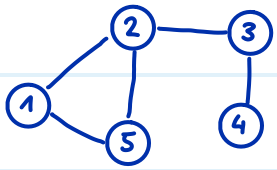
→ $\deg(5) = 2$, $\deg(4) = 1$

Erreicht / reaches: Knoten u erreicht v, falls es einen Weg von u nach v gibt

Zusammenhängend / connected: Jeder Knoten u erreicht jeden Knoten v

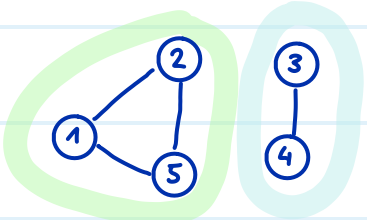
Zusammenhangskomponente (ZHK) / connected component: Eine Äquivalenzklasse der Relation "u erreicht v"

Bsp:



→ zusammenhängend

→ 2 erreicht 4



$ZHK_1 = \{1, 2, 5\}$

$ZHK_2 = \{3, 4\}$

Weg / walk: Eine Folge von $\ell+1$ Knoten $(v_0, v_1, .., v_\ell)$, sodass es alle zwei aufeinanderfolgende Knoten durch eine Kante verbunden sind

Die Länge ℓ ist die Anzahl der Kanten

Knoten und Kanten dürfen mehrfach verwendet werden.

Pfad / path: Ein Weg, der keinen Knoten mehrfach verwendet

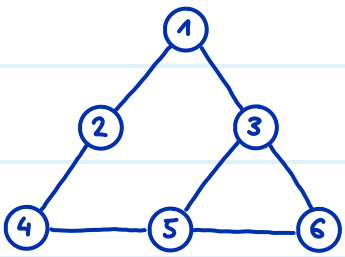
Geschlossener Weg / closed walk: Ein Weg sodass $v_0 = v_\ell$ (Startknoten = Endknoten)

Zyklus / circuit: Ein geschlossener Weg, der keine Kante mehrfach verwendet, und Länge ≥ 1 hat

Kreis / cycle: Ein Zyklus, der keinen Knoten mehrfach verwendet ausser $v_0 = v_\ell$

Achtung: Zyklus ≠ Cycle

Aufgabe:



Wie viele ZHKs gibt es?

Wie viele verschiedene Pfade von 1 nach 6 gibt es?

Wie viele verschiedene Wege von 1 nach 6 gibt es?

Wie viele verschiedene Pfade der Länge 4 von 1 nach 6 gibt es?

Wie viele verschiedene Wege der Länge 3 von 1 nach 3 gibt es?

Wie viele verschiedene Kreise gibt es?

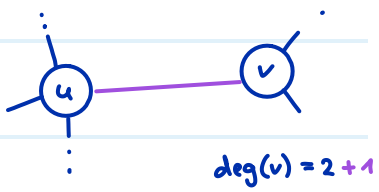
Welche mögliche Längen kann ein Weg von 1 nach 5 haben?

Wie viele verschiedene Zyklen gibt es?

Handschlag-Lemma: $\sum_{v \in V} \deg(v) = 2\,|E|$

Summe aller Knotengrade — 2 · Anzahl Kanten

Intuition: Jedes mal, wenn wir eine neue Kante $\{u, v\}$ hinzufügen, werden $\deg(u)$ und $\deg(v)$ genau um eins grösser, d.h. die Summe aller Knotengerade wird genau um 2 grösser.

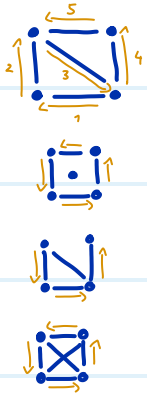


Eulerweg: Ein Weg, der jede Kante genau einmal verwendet

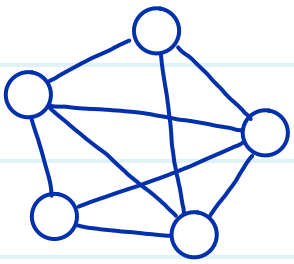
Eulerzyklus: Ein Zyklus, der jede Kante genau einmal verwendet

Hamiltonpfad: Ein Pfad, der jeden Knoten genau einmal verwendet

Hamiltonkreis: Ein Kreis, der jeden Knoten genau einmal verwendet



Aufgabe:

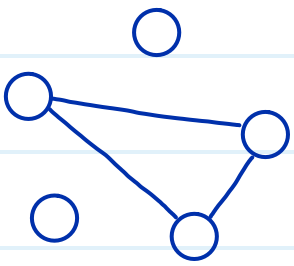


Eulerweg?

Eulerzyklus?

Hamiltonpfad?

Hamiltonkreis?

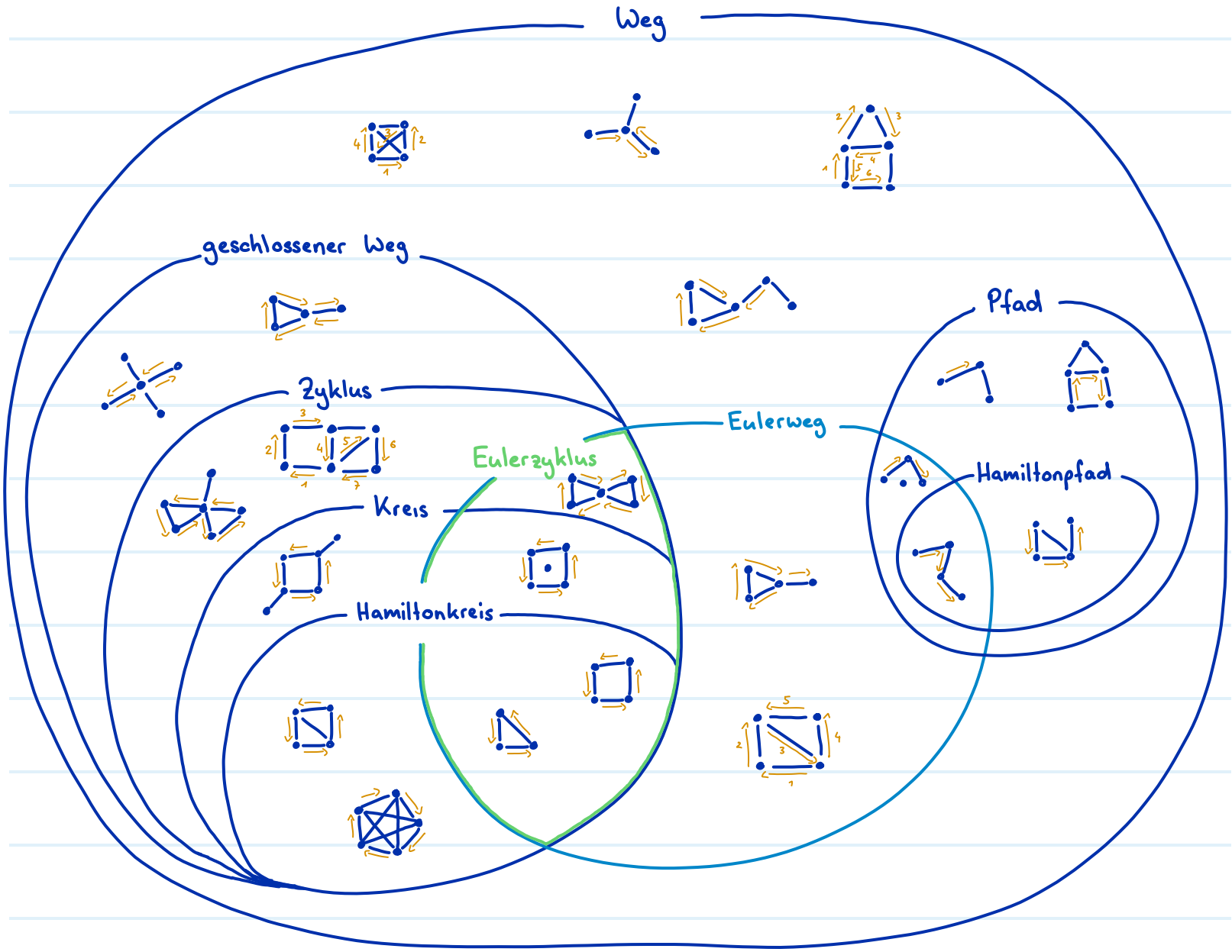


Eulerweg?

Eulerzyklus?

Hamiltonpfad?

Hamiltonkreis?


Weg
geschlossener Weg
Zyklus
Kreis
Hamiltonkreis
Eulerzyklus
Eulerweg
Pfad
Hamiltonpfad

Satz von Euler: Sei G ein zusammenhängender Graph Dann gilt:

G hat einen Eulerzyklus <=> Alle Knotengerade sind gerade

Intuition wir laufen in jeden Knoten gleich oft hinein wie hinaus

Allgemein: Sei G ein Graph Dann gilt:

G hat einen Eulerzyklus <=> Alle Knotengerade sind gerade und

alle Kanten sind in derselben ZHK

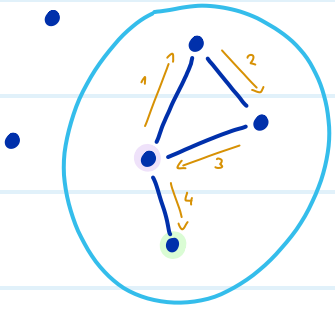
G hat einen Eulerweg <=> Alle Knotengerade sind gerade ausser höchstens zwei und

alle Kanten sind in derselben ZHK

"höchstens zwei": wenn Start ≠ Endknoten, dann laufen wir in den

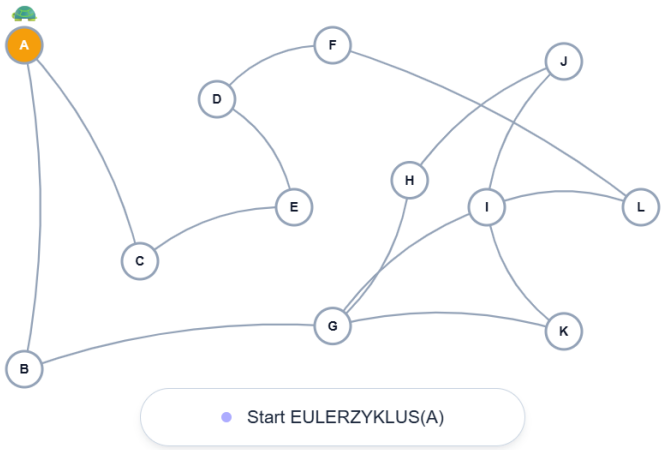
Startknoten einmal weniger hinein als hinaus und in den

Endknoten einmal mehr hinein als hinaus



Alle Kanten in derselben ZHK

Algorithmus von Hierholzer:



EULER TOUR $\mathcal{O}(m)$

```
1  EULERZYKLUS(v_0)
2    W ← FINDEZYKLUS(v_0)
3    laufe W ab; for jeden besuchten Knoten v:
4      if deg(v) > 0 then
5        Z ← FINDEZYKLUS(v)
6        füge Z an der aktuellen Stelle in W ein
7    return W

9  FINDEZYKLUS(v_0)
10   ℓ ← 0
11   repeat
12     if es gibt in E eine zu v_ℓ inzidente Kante e = {v_ℓ, u} then
13       v_{ℓ+1} ← u
14       ℓ ← ℓ + 1
15       lösche e aus E
16     else return W = (v_0, v_1, ..., v_ℓ)
```

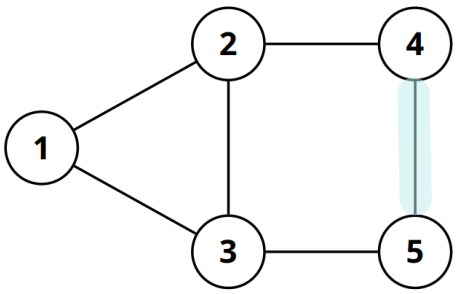
Laufzeit $O(m)$ mit Adjazenzliste

Algorithmus für Hamiltonkreis?

- NP vollständig
- bester bekannter Algorithmus hat exponentielle Laufzeit
- nächstes Semester

Adjazenzmatrix und Adjazenzliste

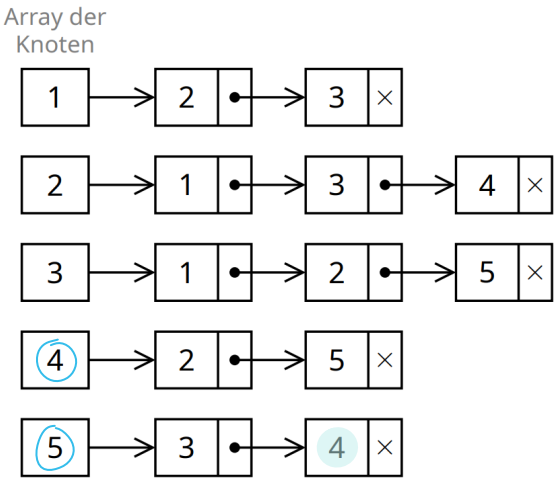
Graph ($n = 5, m = 6$)



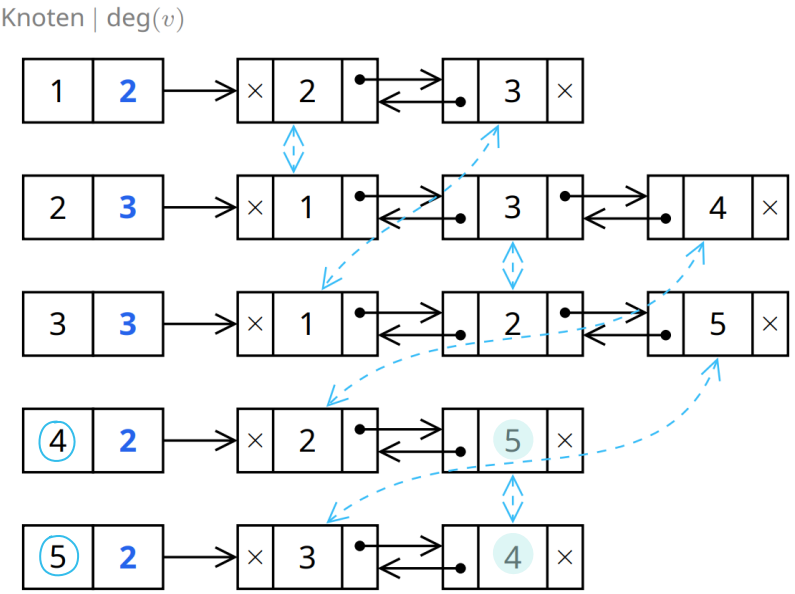
Adjazenzmatrix

$u \setminus v$	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten



Erweiterte Adjazenzlisten



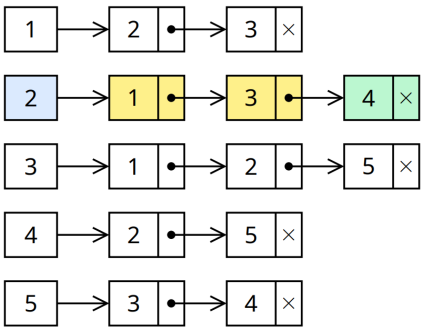
Operation	Adj.-Matrix	Adj.-Listen	erweitert
Ist $\{u, v\} \in E$?	$O(1)$	$O(\deg(u))$	
Finde einen Nachbarn von v	$O(n)$	$O(1)$	
Finde alle Nachbarn von v	$O(n)$	$O(\deg(v))$	
Finde $\deg(v)$	$O(n)$	$O(\deg(v))$	$O(1)$
Lösche Kante $e = \{u, v\}$	$O(1)$	$O(\deg(u) + \deg(v))$	$O(1)$
Füge Kante $e = \{u, v\}$ ein	$O(1)$	$O(\deg(u))$	
Lösche bzw. füge Knoten v ein[4]	$O(n^2)$	$O(n + m)$	$O(n)$

Ist $\{u, v\} \in E$**?** (Example: $\{2, 4\} \in E$?)

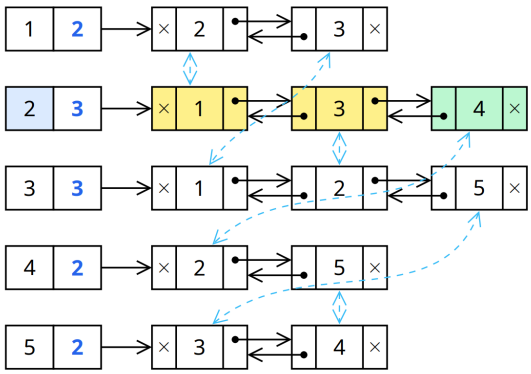
Adjazenzmatrix: $O(1)$

$u \setminus v$	1	2	3	**4**	5
1	0	1	1	0	0
2	1	0	1	**1**	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(\deg(u))$



Erweiterte Adjazenzlisten: $O(\deg(u))$

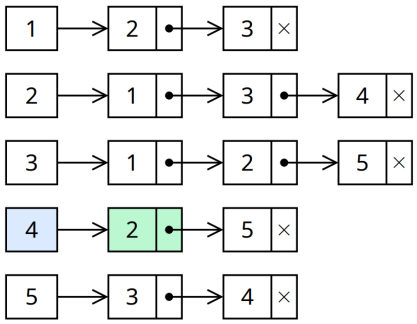


Finde einen Nachbarn von v (Example: $v = 4$)

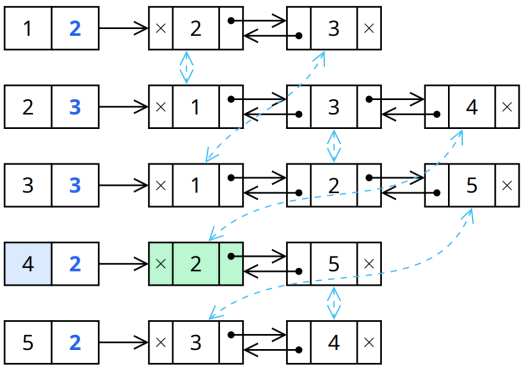
Adjazenzmatrix: $O(n)$

$u \setminus v$	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	**1**	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(1)$



Erweiterte Adjazenzlisten: $O(1)$

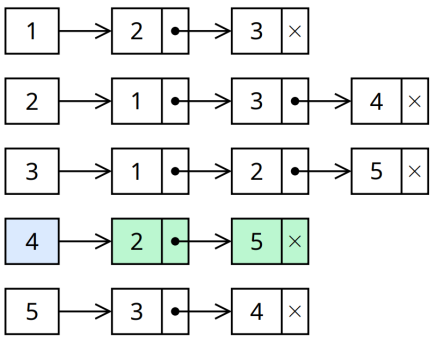


Finde alle Nachbarn von v (Example: $v = 4$)

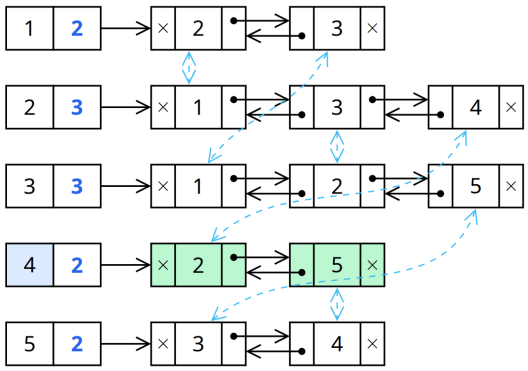
Adjazenzmatrix: $O(n)$

$u \setminus v$	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	**1**	0	0	**1**
5	0	0	1	1	0

Adjazenzlisten: $O(\deg(v))$



Erweiterte Adjazenzlisten: $O(\deg(v))$

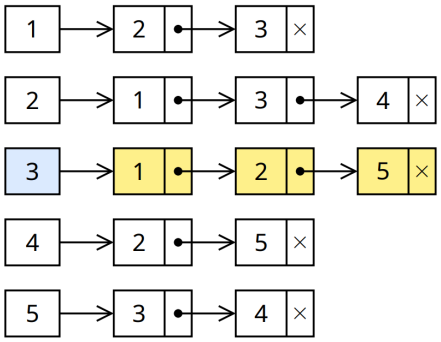


Finde $\deg(v)$ (Example: $v = 3$)

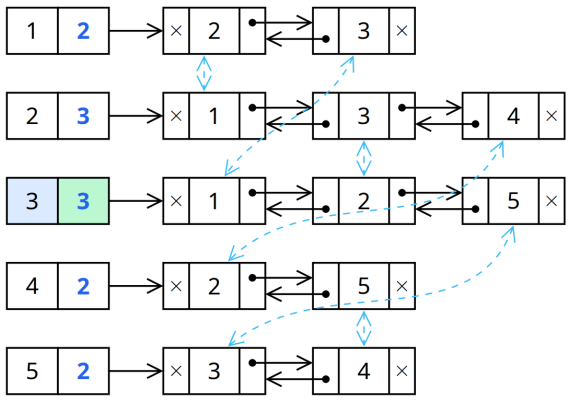
Adjazenzmatrix: $O(n)$

$u \setminus v$	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	**1**	**1**	0	0	**1**
4	0	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(\deg(v))$



Erweiterte Adjazenzlisten: $O(1)$

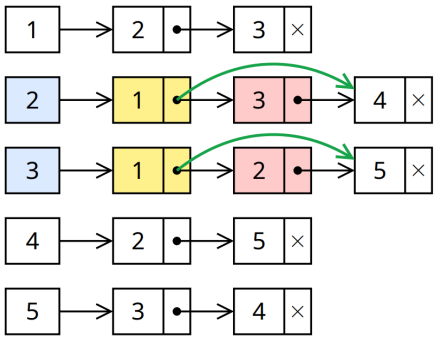


Lösche Kante $e = \{u, v\}$ (Example: Delete $e = \{2, 3\}$)

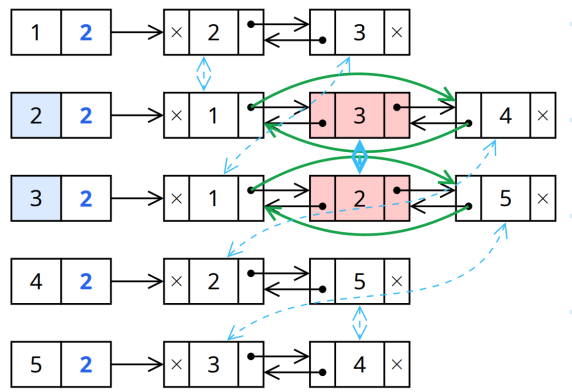
Adjazenzmatrix: $O(1)$

$u \setminus v$	1	**2**	**3**	4	5
1	0	1	1	0	0
2	1	0	**1→0**	1	0
3	1	**1→0**	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(\deg(u) + \deg(v))$



Erweiterte Adjazenzlisten: $O(1)$

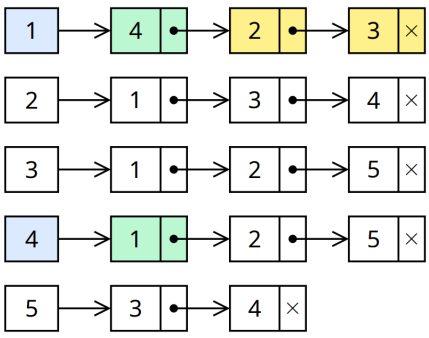


Füge Kante $e = \{u, v\}$ **ein** (Example: Insert $e = \{1, 4\}$)

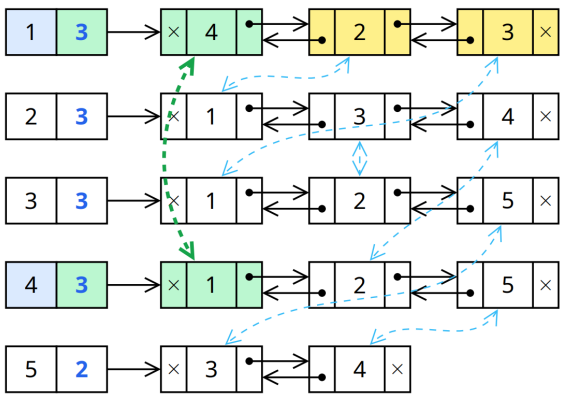
Adjazenzmatrix: $O(1)$

$u \setminus v$	**1**	2	3	**4**	5
1	0	1	1	**0→1**	0
2	1	0	1	1	0
3	1	1	0	0	1
4	**0→1**	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(\deg(u))$



Erweiterte Adjazenzlisten: $O(\deg(u))$

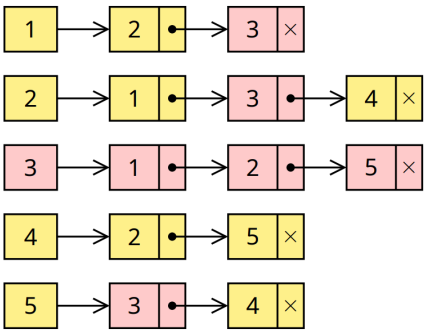


Lösche Knoten v (Example: Delete $v = 3$ and incident edges)

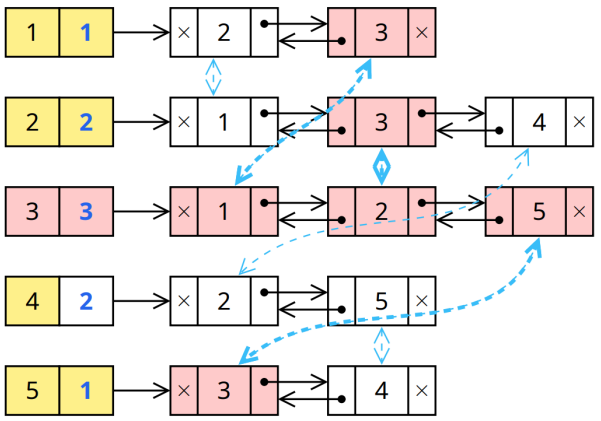
Adjazenzmatrix: $O(n^2)$

$u \setminus v$	1	2	**3**	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

Adjazenzlisten: $O(n+m)$



Erweiterte Adjazenzlisten: $O(n)$

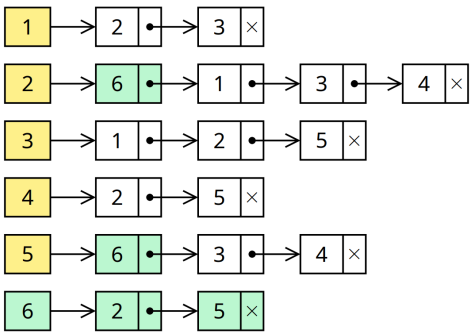


Füge Knoten v **ein** (Example: Insert $v = 6$ with edges $\{2,6\}, \{5,6\}$)

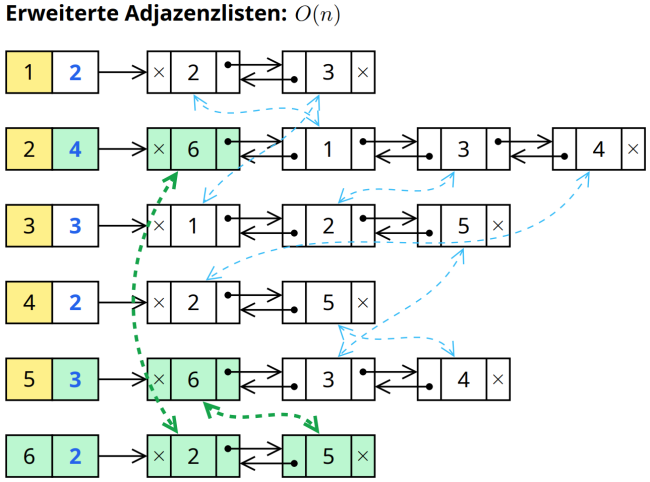
Adjazenzmatrix: $O(n^2)$

$u \setminus v$	1	2	3	4	5	**6**
1	0	1	1	0	0	0
2	1	0	1	1	0	**1**
3	1	1	0	0	1	0
4	0	1	0	0	1	0
5	0	0	1	1	0	**1**
6	0	**1**	0	0	**1**	0

Adjazenzlisten: $O(n)$



Erweiterte Adjazenzlisten: $O(n)$



Tipps für Graph-Beweisaufgaben:

- Immer genaue mathematische Definitionen der Begriffe verwenden
- Beweisstrategien aus DiskMath sind besonders hilfreich:
  - Kontraposition: $A \to B \equiv \neg B \to \neg A$
  - Widerspruchsbeweis: $\neg A \to \bot$
  - Fallunterscheidung
  - Gegenbeispiel

Aufgaben:

Prove or disprove: Jeder Graph, in dem alle Knotengrade gerade sind, ist zusammenhängend.

Prove or disprove: Sei G ein Graph und $a, b, c \in V$ drei verschiedene Knoten von G. Wenn es ein Weg von a nach b, und ein Weg von b nach c gibt, dann gibt es ein Pfad von a nach c.

$A \to B$

Prove or disprove: Wenn ein Graph G einen Eulerzyklus Z hat, sodass Z ein Kreis ist, dann muss G einen Hamiltonkreis haben.

Prove or disprove: Sei G ein Graph mit $n \geq 3$ Knoten. Wenn G zusammenhängend ist und $m = n - 1$ viele Kanten hat, dann muss es einen Knoten mit Grad 1 geben.

Prove or disprove: Jeder Graph, der einen Hamiltonkreis und einen Eulerweg hat, muss auch einen Eulerzyklus haben.

Prove or disprove: Wenn ein Graph G einen Hamiltonpfad besitzt, dann ist G zusammenhängend.

Prove or disprove: Sei G ein zusammenhängender Graph, sodass es für jeden Knoten v einen Eulerweg E_v gibt, der bei v startet. Dann hat G einen Eulerzyklus.

Prove or disprove: Sei G ein zusammenhängender Graph mit $n \geq 4$ Knoten, sodass es für jeden Knoten v einen Eulerweg E_v gibt, der bei v startet. Dann hat G einen Eulerzyklus.

Prove or disprove: Sei G ein Graph mit $n \geq 4$ Knoten, sodass es für jeden Knoten v einen Hamiltonpfad H_v gibt, der bei v startet. Dann hat G einen Hamiltonkreis.

Prove or disprove: Jeder Graph G mit $n \geq 3$ Knoten, in dem jeder Knoten v einen Grad von mindestens $\deg(v) \geq n/2$ hat, muss zusammenhängend sein.